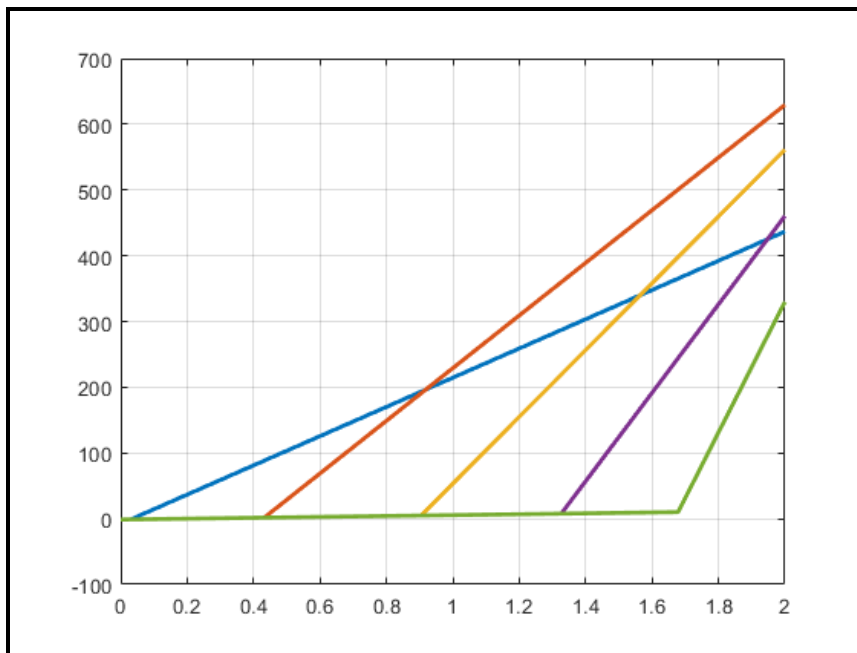


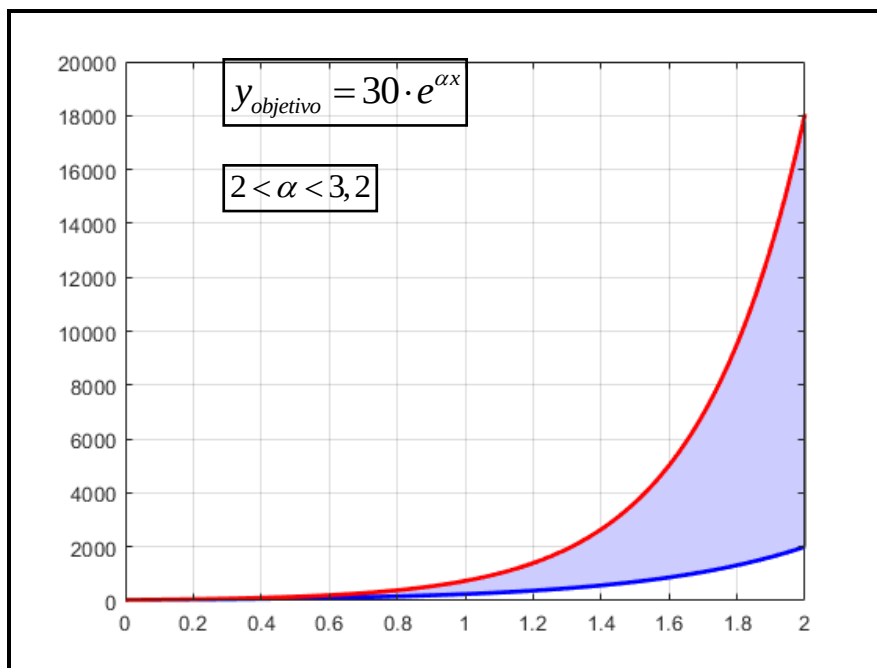
Aproximación de la función objetivo con una combinación lineal de coeficientes enteros para 5 funciones base, realizado con una Red Neuronal entrenada.

Para aproximar una función objetivo tipo J con una combinación lineal de funciones base, se usó la técnica de la predicción de dicha función mediante redes neuronales. Los datos de entrenamiento se logran buscando los valores óptimos con el método de los Multiplicadores de Lagrange (reales). Luego se entrenó a una Red Neuronal con datos generados por el método numérico (reales). Luego se compararon las predicciones del método numérico y los de la red neuronal (reales). Luego se redondearon las predicciones de la red neuronal (enteros), y se generaron las curvas de aproximación finales.

Funciones Base nuevas



Rango de variación de las funciones objetivo para aproximaciones con $R^2 > 0,98$



Estrategia de generalización de predicciones para la Red Neuronal:

- **Antes:** entrada = escalar α .
- **Ahora:** entrada = **la Curva J completa muestreada** en la malla x (un vector de la misma longitud que la malla del problema, no un solo número).

Esto es lo que le da a la red neuronal la capacidad de generalizar — ve la curva entera, no un resumen de 1 número.

Detalle técnico importante: las curvas crecen exponencialmente (~ 30 a $\sim 30 \cdot e^{6.4} \approx 18,000$), así que alimentar los valores crudos a la red va a generar una escala enorme y mal condicionada para el entrenamiento. Trabajamos con **$\log(b(x))$** como entrada — para aplanar el rango dinámico y, para las curvas base (que ya vienen de exponenciales), vuelve la relación mucho más "amigable" para la red.

Generación de Curvas J no exponenciales para entrenamiento

Generación de curvas como combinaciones convexas, punto a punto, de varias exponenciales con distintos α , usando un peso $w(x)$ que varía suavemente a lo largo de x (no constante). Es decir:

Definición formal

Para cada muestra de entrenamiento se sortea un número de componentes k y, para cada componente, un exponente:

$$k \sim \mathcal{U}\{2, 3, 4, 5, 6\} \quad (\text{entero discreto uniforme})$$

$$\alpha_i \sim \mathcal{U}[2.0, 3.2], \quad i = 1, \dots, k$$

Cada componente define una curva exponencial base:

$$g_i(x) = 30 e^{\alpha_i x}, \quad i = 1, \dots, k$$

Pesos suaves dependientes de x

Se sortea un número de nodos de control $n_c \sim \mathcal{U}\{3, \dots, 8\}$, ubicados en una malla equiespaciada $x_1, \dots, x_{n_c} \in [0, 2]$. Para cada componente i se sortean logits en esos nodos:

$$\ell_i(x_j) \sim \mathcal{N}(0, 2^2), \quad j = 1, \dots, n_c$$

y se interpolan con un spline cúbico para obtener una función continua $\ell_i(x)$ en toda la malla. Los pesos se definen mediante un softmax **evaluado punto a punto** entre las k componentes:

$$w_i(x) = \frac{e^{\ell_i(x)}}{\sum_{j=1}^k e^{\ell_j(x)}}, \quad i = 1, \dots, k$$

lo cual garantiza $w_i(x) \geq 0$ y $\sum_{i=1}^k w_i(x) = 1$ para todo x .

Curva J resultante

$$b(x) = \sum_{i=1}^k w_i(x) g_i(x) = \sum_{i=1}^k w_i(x) \cdot 30 e^{\alpha_i x}$$

Además, una fracción del 15% de las curvas se genera con $k = 1$ (exponencial pura, $b(x) = 30e^{\alpha x}$ con $\alpha \sim \mathcal{U}[2.0, 3.2]$), para anclar el caso ya validado originalmente.

Garantía de envolvente (para la demostración en el artículo)

Como $30e^{\alpha x}$ es monótona creciente en α para $x \geq 0$, se tiene $g_i(x) \in [30e^{2x}, 30e^{3.2x}]$ para todo i y todo $x \geq 0$. Como $b(x)$ es una combinación convexa (pesos no negativos que suman 1) de valores dentro de ese intervalo, se sigue que:

$$30 e^{2x} \leq b(x) \leq 30 e^{3.2x}, \quad \forall x \in [0, 2]$$

independientemente de k , de los α_i elegidos, o de la forma de $w_i(x)$. Esta es una propiedad **matemática exacta**, no una verificación empírica — la verificación de "0 violaciones" que corrió el script es una confirmación, no la prueba en sí.

La función aproximada es del siguiente tipo:

$$y_{\text{aprox}}(x) = c_1 \cdot b_1(x) + c_2 \cdot b_2(x) + c_3 \cdot b_3(x) + c_4 \cdot b_4(x) + c_5 \cdot b_5(x)$$

Donde los c_i son los coeficientes dados por los diferentes métodos de aproximación para cada **Curva J** correspondiente de la la función objetivo, y donde los $b_i(x)$ son las distintas curvas base.

Arquitectura de la Red Neuronal

```
model = Sequential([  
  
    Dense(256, activation="relu", kernel_regularizer=l2(L2_REGULARIZATION),  
        input_shape=(n_puntos,)),  
    BatchNormalization(),  
    Dropout(DROPOUT_RATES[0]),  
  
    Dense(256, activation="relu", kernel_regularizer=l2(L2_REGULARIZATION)),  
    BatchNormalization(),  
    Dropout(DROPOUT_RATES[1]),  
  
    Dense(128, activation="relu", kernel_regularizer=l2(L2_REGULARIZATION)),  
    BatchNormalization(),  
    Dropout(DROPOUT_RATES[2]),  
  
    Dense(5, activation="linear"),  
])
```

Fig. 1 – Arquitectura de la Red Neuronal – Código Python

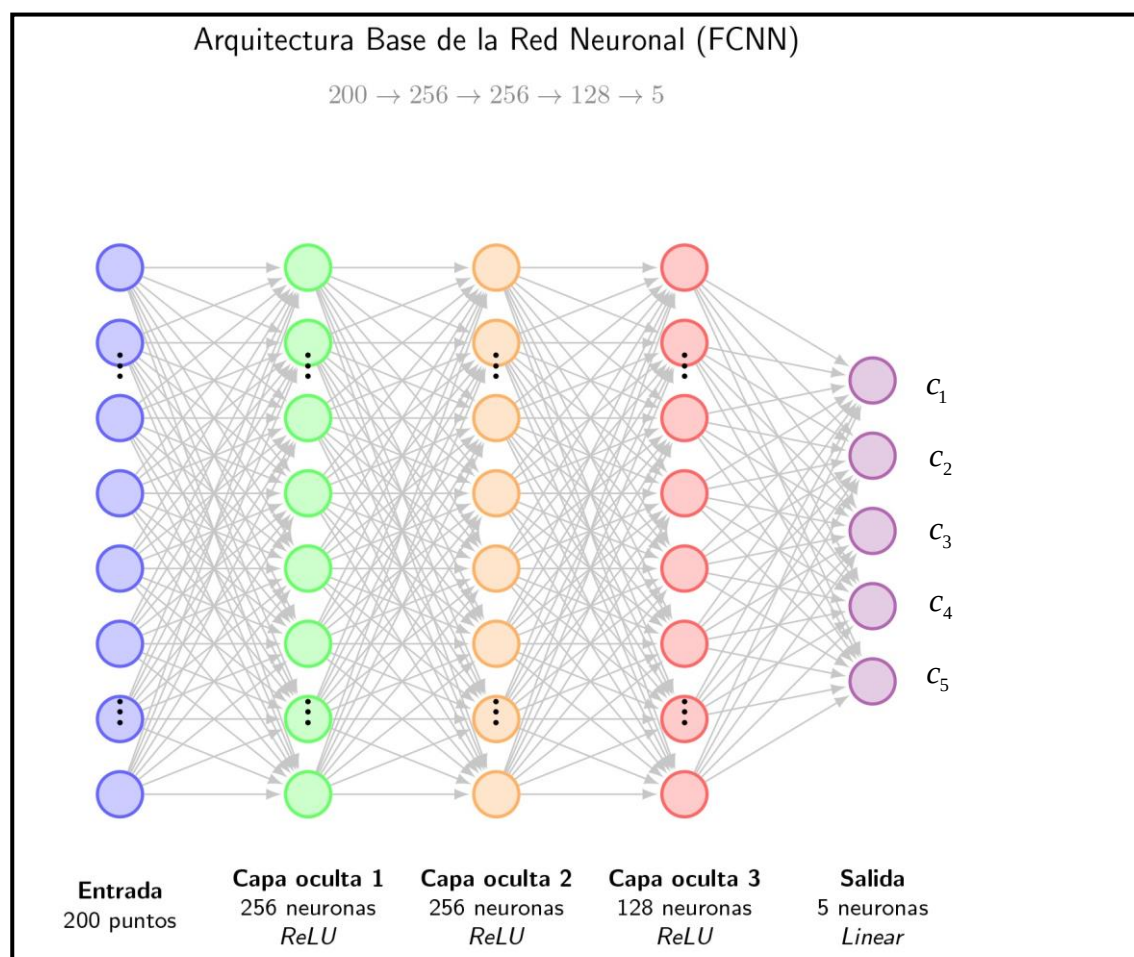


Fig. 2 – Arquitectura de la Red Neuronal

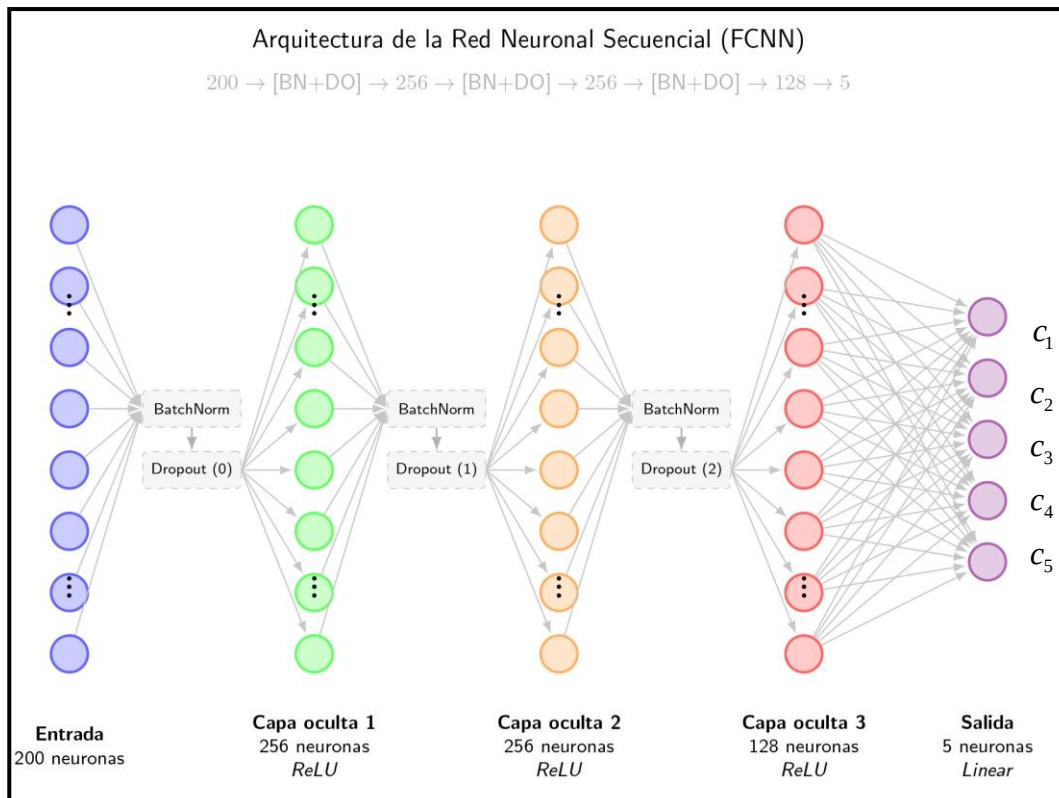


Fig. 3 – Arquitectura de la Red Neuronal – Fase de Entrenamiento

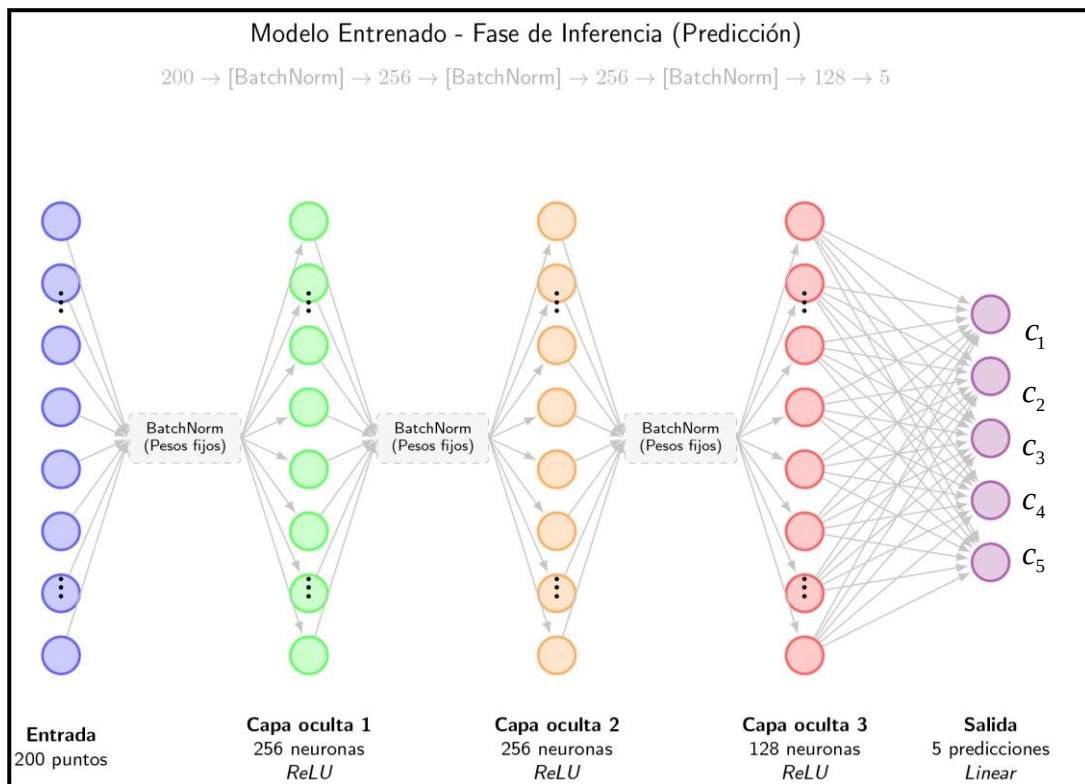
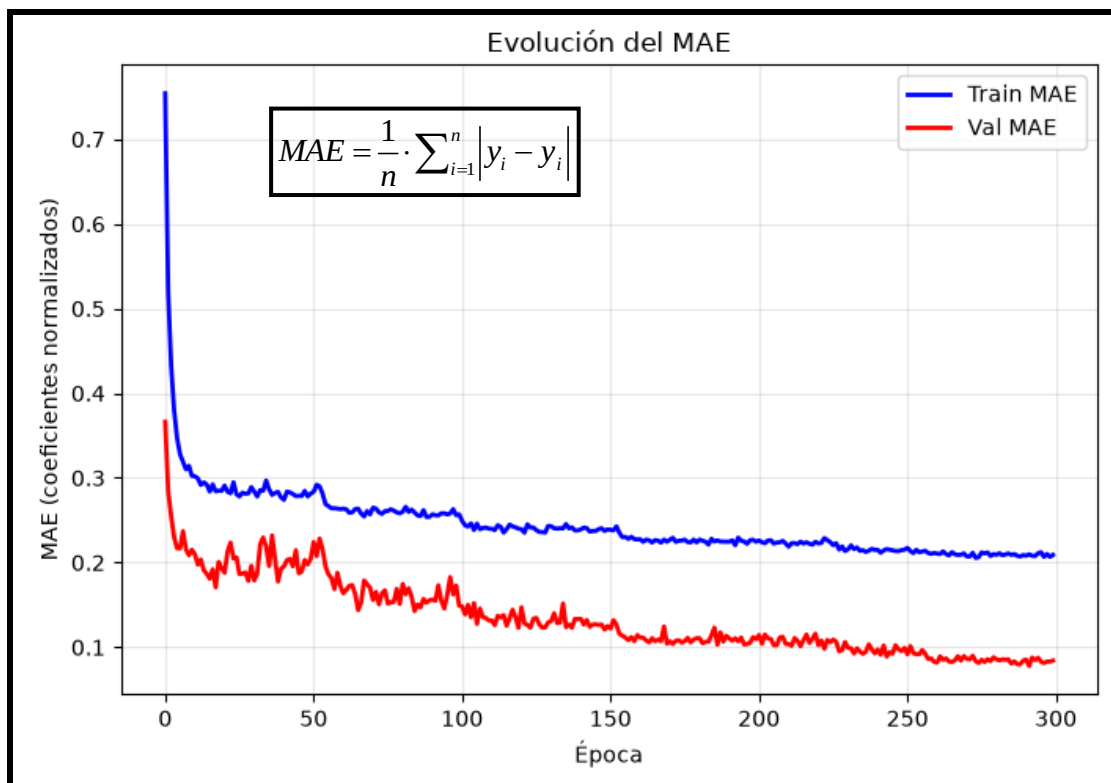
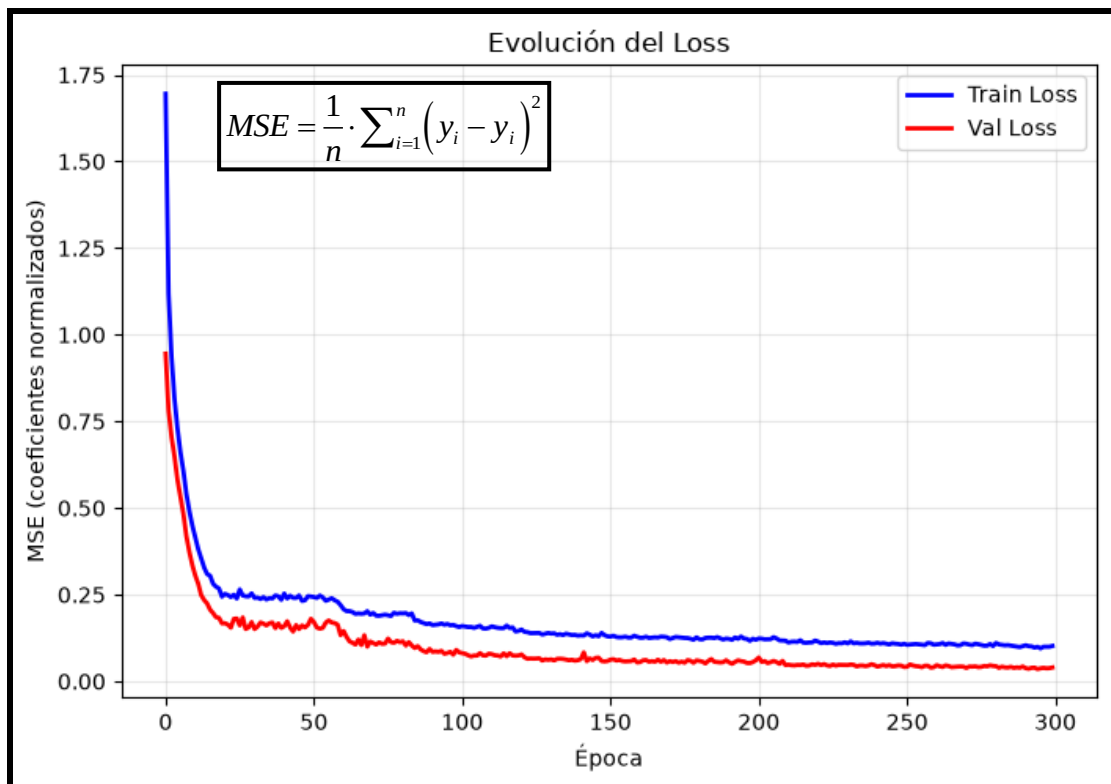


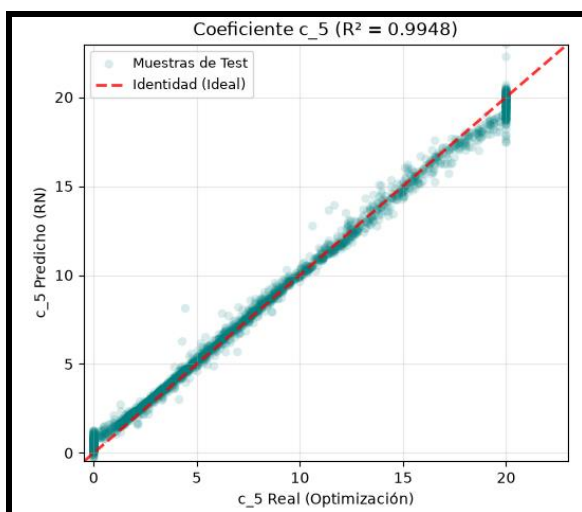
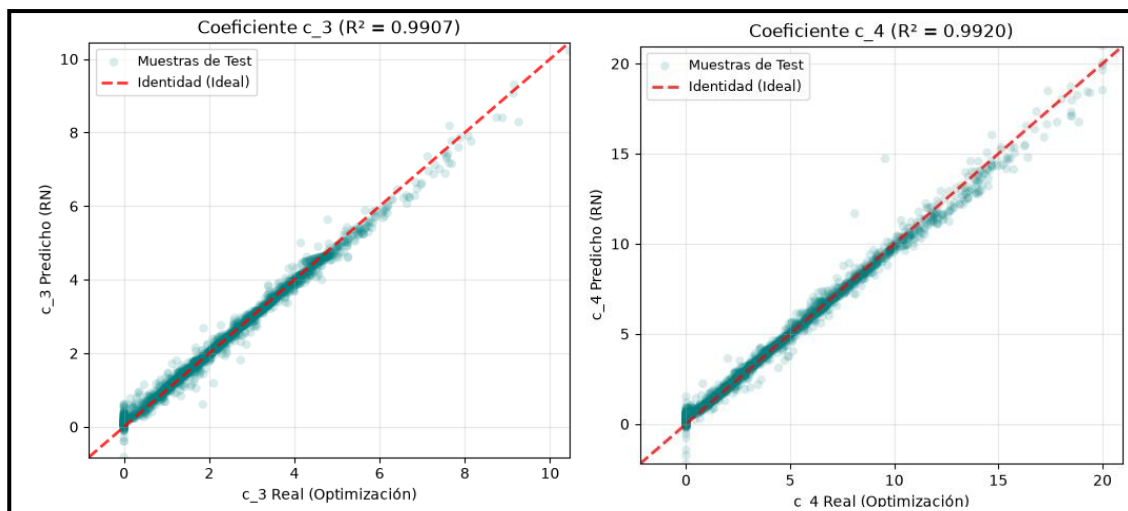
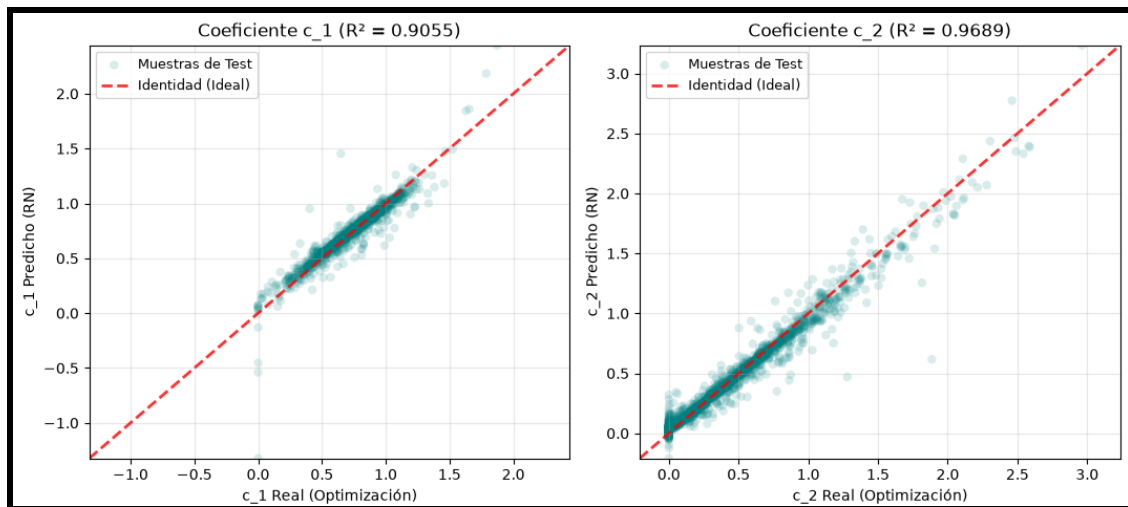
Fig. 4 – Arquitectura de la Red Neuronal – Modelo Entrenado

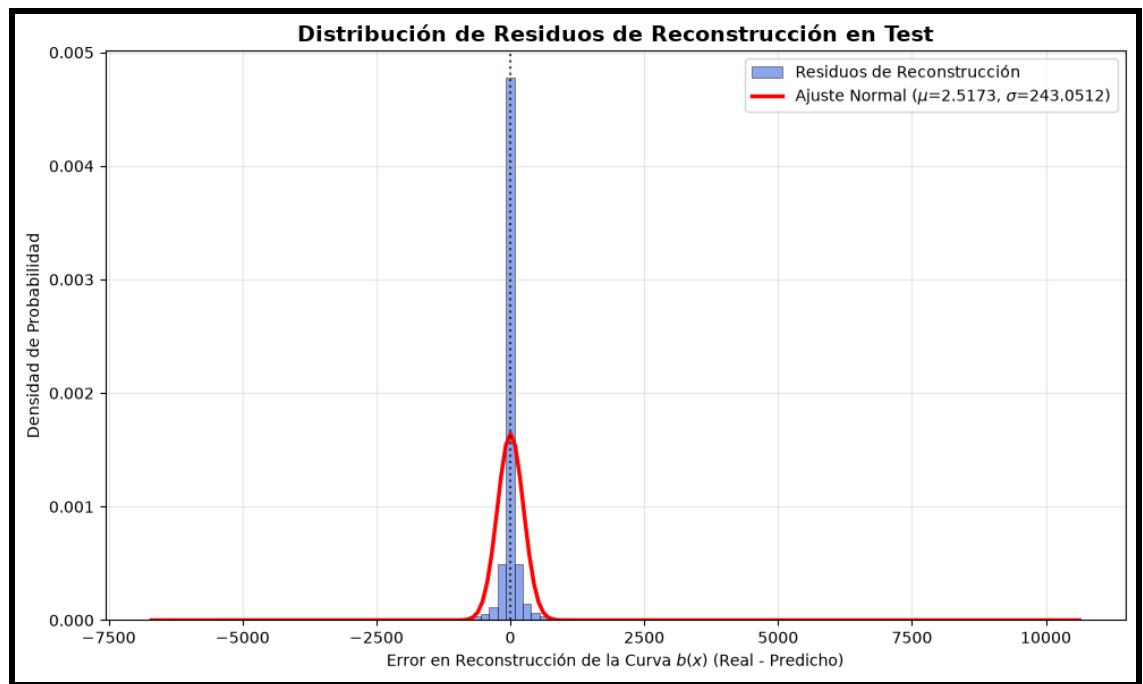
Métricas del entrenamiento y validación de la Red Neuronal

No hay overfitting ya que las pérdidas en el entrenamiento y en el test disminuyen



Comparativa de Coeficientes: Real vs. Predicho por RN





Predicciones de la RN para una curva J de entrada

